Live Like Sync and Notification Verification

Date: 2026-06-02
Project: Yenkasa
Release target: Android versionName 5.3, versionCode 60

Scope

This report documents the livestream like synchronization audit, community post notification work, livestream start notification work, notification operational intelligence events, and verification evidence completed for the 2026-06-02 batch.

The work was done carefully around the existing livestream operational intelligence integration documented in /Users/kofibright/Desktop/YenkasaResearch/livestream-operational-intelligence-integration-2026-06-02.pdf. Existing livestream OIL events and socket behavior were preserved.

Deliverables Status

- Like sync bug fixed: Complete in code path. Backend accepts like aliases, broadcasts new_like, and emits aggregate like counts. Android listener updates the UI immediately and animates hearts.
- Community post notifications implemented: Complete in code path. Joined community members receive COMMUNITY_POST_CREATED notifications; author, blocked users, suspended users, and recently notified duplicates are excluded.
- Livestream start notifications implemented: Complete in code path. Community streams notify community members; creator streams notify followers.
- Android frontend verified: Compile verification passed with ./gradlew :app:compileDebugKotlin.
- Web frontend verified: Production build passed with npm run build.
- OIL events generated: Complete in code path for STREAM_STARTED, COMMUNITY_POST_CREATED, NOTIFICATION_SENT, NOTIFICATION_OPENED, and NOTIFICATION_DISMISSED.
- End-to-end test evidence provided: Automated backend, Android compile, web build, and code-path evidence are included below. A physical three-device host/viewer push-delivery run was not performed in this terminal session.

Priority 1: Livestream Like Sync

Audited search terms:

- send_like
- new_like
- live_reaction
- likeStream
- streamLike
- reaction
- heart

Verified flow:

1. Frontend Like Button: Android LiveStreamActivity sends a live reaction from the heart button.
2. Socket Emit: Android emits live_reaction with stream/user context and a client event id.
3. Socket Handler: Backend handles live_reaction, live_like, send_like, streamLike, and likeStream.
4. Backend Event Processing: Backend deduplicates incoming reaction events, increments an in-memory stream reaction count, and emits livestream operational event STREAM_LIKE.
5. Room Broadcast: Backend broadcasts both live_reaction and required compatibility event new_like to the livestream room.
6. Frontend Listener: Android listens for both live_reaction and new_like.

7. UI Update: Android updates the like count from likeCount, totalLikes, or reactionCount immediately.
8. Heart Animations: Android runs the heart/reaction animation for incoming non-duplicate events and runs optimistic local animation for the sender.
9. Simultaneous Likes: Backend aggregate counting is per stream and broadcasts the updated count with each accepted event, allowing multiple viewers to like concurrently.

Expected device behavior:

- Viewer A likes: Host receives, Viewer A reconciles count, Viewer B receives.
- Viewer B likes: Host receives, Viewer A receives, Viewer B reconciles count.

Finding:

- The original weak point was compatibility and count propagation. Existing comments/live events worked, but like listeners and emit names could miss each other and there was no reliable aggregate count in the broadcast. The backend now broadcasts new_like with aggregate count data, and Android listens to both expected event names.

Priority 2: Community Post Notifications

Implemented event:

```
{
  "type": "COMMUNITY_POST_CREATED",
  "communityId": "...",
  "communityName": "...",
  "postId": "...",
  "authorId": "...",
  "authorName": "...",
  "timestamp": "..."
}
```

Notification rules:

- Targets only users who joined the community.
- Excludes the post author.
- Excludes users blocked by or blocking the author.
- Excludes users with suspended accounts.
- Excludes users who blocked that community.
- Suppresses very recent duplicate notifications for the same author/community/recipient.

Notification copy:

- Title: New post in ATU Students
- Body: Bright Kofi shared a new post.
- Action target: community route with community id and post id.

Implementation notes:

- communityPostNotification.service.js now exposes buildCommunityPostNotificationData so the payload is unit tested without sending live notifications.
- dispatchCommunityPostNotifications now returns stats, making operational verification easier.
- Payload includes the required event fields in push data and notification metadata.

Priority 3: Livestream Start Notifications

Implemented event:

```
{
  "type": "STREAM_STARTED",
  "streamId": "...",
  "hostId": "...",
  "hostName": "...",
  "communityId": "...",
  "communityName": "...",
  "title": "...",
  "timestamp": "..."
}
```

Notification rules:

- If the stream belongs to a community, notify eligible community members.
- If the stream does not belong to a community, notify eligible followers of the creator.
- Excludes the host.
- Excludes blocked users.
- Excludes suspended users.
- Excludes users who blocked the community.
- Suppresses duplicate STREAM_STARTED notifications for the same stream/recipient.

Notification copy:

- Creator stream title: Bright Kofi is now live.
- Creator stream body: Join livestream now.
- Community stream title: A livestream has started in ATU Students.
- Community stream body: Watch now.

Implementation notes:

- livestreamStartNotification.service.js resolves whether the stream belongs to a community.
- The livestream socket host-ready flow queues notifications after the stream is marked live and broadcast as started.
- buildLivestreamStartNotificationData is exported for direct payload testing.

Priority 4: OIL Integration

Implemented notification operational events:

- STREAM_STARTED
- COMMUNITY_POST_CREATED
- NOTIFICATION_SENT
- NOTIFICATION_OPENED
- NOTIFICATION_DISMISSED

Verified routing:

- Events publish through the existing YME publisher with notification operational schema metadata.
- YME ingestion aliases include the new notification/community lifecycle events.
- Intelligence event publisher supports the new event types and maps metadata for downstream analytics.
- Recommendation and interest services include weights/triggers for notification/community engagement.
- Notification creation emits NOTIFICATION_SENT.
- Android/web mark-open flows emit NOTIFICATION_OPENED.

- Android swipe-dismiss flow emits NOTIFICATION_DISMISSED.

Frontend Verification

Android:

- Version updated to versionName = "5.3" and versionCode = 60.
- Notification adapter recognizes community_post_created and stream_started.
- Notification open/dismiss calls include interaction intent.
- Livestream UI binds and updates the like count text.
- Livestream UI listens for live_reaction and new_like.
- Verification command passed: ./gradlew :app:compileDebugKotlin.

Web:

- Notification routing recognizes community_post_created.
- Notification routing maps stream-start/live targets without breaking current web route constraints.
- Notification read calls include interaction=opened.
- Verification command passed: npm run build.

Automated Test Evidence

Backend command:

npm test

Result:

- 18 tests passed.
- 0 failed.

Relevant tests:

- Livestream realtime service tracks aggregate reaction count per stream.
- Intelligence publisher converts community post notification events.
- Intelligence publisher converts notification lifecycle events.
- Community post notification payload opens community and carries required event context.
- Livestream start notification payload uses community copy when stream belongs to community.
- Livestream start notification payload falls back to follower copy without community.

Syntax checks:

- node -c services/notificationOperationalEvents.service.js: passed.
- node -c services/communityPostNotification.service.js: passed.
- node -c services/livestreamStartNotification.service.js: passed.
- node -c src/services/livestream/livestream.socket.js: passed.

Frontend commands:

- ./gradlew :app:compileDebugKotlin: passed.
- npm run build: passed.

Remaining Manual Verification

A real three-device live run should still be performed before release signoff:

- Host Device

- Viewer A
- Viewer B

Manual like matrix:

- Viewer A likes: Host OK, Viewer A OK, Viewer B OK.
- Viewer B likes: Host OK, Viewer A OK, Viewer B OK.

Manual notification matrix:

- User A posts in ATU Students: User B receives, User C receives, User A does not.
- Community livestream starts: eligible community members receive.
- Creator livestream starts outside a community: eligible followers receive.
- Tapping notification records NOTIFICATION_OPENED.
- Dismissing notification records NOTIFICATION_DISMISSED.
- Push delivery requires live FCM/OneSignal credentials and reachable devices.

Key Files Implemented or Verified

- app/build.gradle.kts
- app/src/main/java/xyz/yenkasa/app/adapter/NotificationAdapter.kt
- app/src/main/java/xyz/yenkasa/app/network/ApiService.kt
- app/src/main/java/xyz/yenkasa/app/ui/LiveStreamActivity.kt
- app/src/main/java/xyz/yenkasa/app/ui/UserNotificationsActivity.kt
- app/src/main/res/layout/activity_live_stream.xml
- app/src/main/res/values/strings.xml
- yenkasa-web/src/api/notifications.js
- yenkasa-web/src/utils/notificationRouting.js
- services/communityPostNotification.service.js
- services/livestreamStartNotification.service.js
- services/notification.service.js
- services/notificationOperationalEvents.service.js
- routes/notifications.routes.js
- src/intelligence/services/eventPublisher.service.js
- src/services/livestream/livestream.service.js
- src/services/livestream/livestream.socket.js
- src/yme/config/yme.config.js
- src/yme/services/consolidation.service.js
- src/yme/services/eventGuard.service.js
- src/yme/services/eventIngestion.service.js
- src/yme/services/importanceScoring.service.js
- src/yme/services/interestExtraction.service.js
- src/yme/services/recommendationSignals.service.js
- tests/intelligenceEventPublisher.test.js
- tests/livestreamOperationalEvents.test.js
- tests/notificationFanoutPayloads.test.js

Current uncommitted files from this verification pass:

- app/build.gradle.kts
- services/communityPostNotification.service.js
- services/livestreamStartNotification.service.js
- tests/notificationFanoutPayloads.test.js
- docs/live-like-community-stream-notification-verification-2026-06-02.md
- docs/live-like-community-stream-notification-verification-2026-06-02.pdf

Final Finding

The notification and livestream like sync paths are implemented and verified by automated tests, build checks, and code-path inspection. The only remaining release-risk item is physical device verification for real-time socket fanout and push delivery, because that requires live devices and production-like notification credentials.